

Práctica #1 Fuerza bruta y Backtracking

Mateo Gómez Giraldo

Simón Mazo Gómez

Johan Alejandro Peña Rivera

Universidad EAFIT

Análisis y diseño de algoritmos

Carlos Alberto Álvarez Henao

31 de agosto de 2026

1. Introducción

El presente trabajo analiza y compara los paradigmas de **Fuerza Bruta** y **Backtracking** aplicados a un problema relacionado con la generación y búsqueda de contraseñas sobre espacios finitos de cadenas. El problema se aborda desde dos perspectivas complementarias: en el módulo de **Fuerza Bruta** se busca determinar el costo de encontrar una contraseña mediante la enumeración sistemática de todas las combinaciones posibles y su posterior verificación mediante SHA-256; mientras que en el módulo de **Backtracking** se busca construir contraseñas que cumplan determinadas políticas de seguridad, evitando explorar aquellas ramas del espacio de búsqueda que ya no pueden conducir a una solución válida.

La motivación principal del trabajo es comprender, a partir de una implementación y experimentación propias, cómo el tamaño del espacio de búsqueda afecta el rendimiento de un algoritmo y cómo el uso de información parcial puede reducir significativamente la cantidad de estados explorados. En particular, se busca contrastar el crecimiento exponencial esperado para la enumeración exhaustiva con el comportamiento obtenido experimentalmente, así como cuantificar el beneficio de utilizar poda en **Backtracking**.

El documento presenta inicialmente el contexto y la fundamentación teórica de ambos paradigmas, para posteriormente formalizar los espacios de búsqueda correspondientes a cada módulo. Luego se describen las decisiones de diseño, el pseudocódigo y los aspectos relevantes de la implementación. Finalmente, se presenta el análisis teórico de complejidad, los casos de prueba, los resultados experimentales y su respectivo análisis, incluyendo las comparaciones entre **Fuerza Bruta** y búsqueda por diccionario, así como entre **Backtracking** con poda y una exploración exhaustiva sin poda.

2. Contexto del problema

El módulo de **fuerza bruta** del trabajo plantea un escenario en el que se conoce únicamente el hash SHA-256 asociado a una contraseña sintética, pero no la contraseña original. El procedimiento utilizado consiste en generar sistemáticamente candidatos pertenecientes a un espacio de cadenas determinado, calcular el hash de cada candidato y compararlo con el hash objetivo. La estrategia de **Fuerza Bruta** resulta apropiada para este escenario porque no utiliza información adicional para

descartar candidatos: cada combinación perteneciente al espacio debe considerarse hasta encontrar la contraseña o agotar completamente las posibilidades.

El módulo de **backtracking** aborda el problema desde una perspectiva diferente. En lugar de buscar una contraseña específica, se desea generar o contar contraseñas que cumplan una política de seguridad determinada. Esta política establece condiciones como cantidades mínimas de letras minúsculas, mayúsculas, dígitos y símbolos, además de prohibir la aparición de caracteres idénticos consecutivos. Generar primero todas las cadenas posibles y verificar posteriormente cuáles cumplen las condiciones equivaldría a realizar una búsqueda exhaustiva. Por ello, se utiliza **Backtracking**, construyendo la contraseña carácter por carácter y verificando durante la construcción si el prefijo actual todavía puede llegar a convertirse en una solución válida. Cuando esto deja de ser posible, se descarta inmediatamente la rama correspondiente.

3. Fundamentación teórica

La **Fuerza Bruta** consiste en explorar sistemáticamente todos los candidatos pertenecientes a un espacio de soluciones, sin utilizar propiedades del problema para reducir dicho espacio. En este trabajo, si el alfabeto tiene tamaño $|\Sigma|$ y las contraseñas tienen longitud n , el número de candidatos posibles es $|\Sigma|^n$. Por ejemplo, el alfabeto A1 contiene 26 símbolos, por lo que una contraseña de longitud 6 produce un espacio de 26^6 candidatos; de manera similar, el alfabeto A2 contiene 36 símbolos y una contraseña de longitud 5 produce 36^5 posibilidades. El algoritmo debe generar cada candidato exactamente una vez y verificarlo mediante su hash SHA-256.

Backtracking también realiza una búsqueda sobre un espacio potencialmente exponencial, pero organiza dicha búsqueda como un árbol de estados parciales. En este caso, cada estado representa un prefijo de la contraseña y cada nivel del árbol corresponde a agregar un carácter. La diferencia fundamental es la utilización de una función de factibilidad: antes de continuar expandiendo un prefijo, se determina si todavía existe alguna posibilidad de completar la contraseña respetando las restricciones. Si un prefijo ya no puede conducir a una solución válida, se poda la rama y no se generan sus descendientes. De esta manera, el algoritmo no necesita construir todas las cadenas completas para determinar cuáles cumplen la política.

Esta fundamentación se aplica directamente a la instancia del equipo. A partir de la semilla 1405 —creada a partir de la suma de los códigos ascii de los apellidos de los integrantes—, los parámetros de la política de **Backtracking** son un mínimo

de 3 letras minúsculas, 2 mayúsculas, 2 dígitos y 1 símbolo, para una longitud total de 8 caracteres. Por tanto, la política exige exactamente las ocho posiciones para satisfacer los mínimos de composición, además de impedir caracteres idénticos consecutivos. Estas restricciones permiten realizar poda durante la construcción, por ejemplo, cuando el número de posiciones restantes resulta insuficiente para alcanzar alguno de los mínimos requeridos. En consecuencia, el beneficio de **Backtracking** depende directamente de qué tan pronto estas condiciones permitan identificar que una rama es inviable.

4. Modelamiento

Para el módulo de **Fuerza Bruta**, el espacio de búsqueda se define a partir de un alfabeto Σ y una longitud n . Cada estado terminal corresponde a una cadena completa de longitud n , mientras que el conjunto de candidatos tiene cardinalidad $|\Sigma|^n$. En la implementación se consideran dos alfabetos: A1, compuesto por las 26 letras minúsculas del alfabeto inglés, y A2, compuesto por las 26 letras minúsculas más los 9 números [1-9], para un total de 35 símbolos. Las longitudes establecidas para las pruebas son de 3 a 6 caracteres para A1 y de 3 a 5 caracteres para A2. El algoritmo recibe como entrada el hash objetivo, el alfabeto y la longitud máxima correspondiente, y genera candidatos de manera sistemática hasta encontrar aquel cuyo hash coincide con el objetivo o hasta agotar el espacio de búsqueda.

Para el módulo de **Backtracking**, el espacio inicial está determinado por un alfabeto de 69 símbolos, compuesto por letras minúsculas, letras mayúsculas, dígitos y los símbolos !, @, #, \$ y %. Una solución tiene una longitud fija de $n = 8$ caracteres para la política principal. Un estado parcial se representa mediante el prefijo construido hasta el momento y la información necesaria para comprobar las restricciones, como la cantidad de minúsculas, mayúsculas, dígitos y símbolos utilizados, además del último carácter agregado. El estado inicial corresponde a la cadena vacía y cada transición consiste en agregar un carácter del alfabeto. Los estados terminales corresponden a cadenas de longitud n , que se aceptan únicamente si cumplen completamente la política.

En la instancia del equipo, la semilla 1405 determina los valores mínimos de la política: $\text{minLower} = 3$, $\text{minUpper} = 2$, $\text{minDigit} = 2$ y $\text{minSymbol} = 1$. La suma de estos requisitos es igual a 8, por lo que una contraseña de longitud 8 debe cumplir exactamente estos mínimos de composición. Adicionalmente, se mantiene la restricción de que no pueden aparecer dos caracteres idénticos consecutivamente. El árbol completo tendría, conceptualmente, 69^k estados en el nivel k , aunque el

Backtracking no necesariamente visita todos ellos debido a la poda. Por esta razón, además del tamaño teórico del espacio, resulta relevante medir experimentalmente cuántos nodos son realmente visitados y qué porcentaje del espacio se logra descartar.

5. Diseño algorítmico

Para la solución del módulo de **Fuerza Bruta**, se utilizó la estructura de árbol para generar todas las posibles combinaciones de manera recursiva, con factor de recursividad $|\Sigma|^n$. Cuando el algoritmo llega a una de las hojas (la contraseña generada cumple con el tamaño especificado), se compara con el hash objetivo para verificar un posible éxito; de esta manera, se evitan comparaciones innecesarias de contraseñas que aún no cumplan con el tamaño; cuando el algoritmo encuentra la cadena objetivo, se corta la recursividad y se finaliza la función (nótese que el hallazgo de esta no está acotado por ninguna restricción que pueda minimizar el proceso).

Para la solución del módulo de **Backtracking**, se diseñó una función de factibilidad que determina, a partir de un prefijo parcial, si aún es posible completar una contraseña válida sin violar la política. Esta función calcula, para cada tipo de carácter (minúscula, mayúscula, dígito, símbolo), cuántas unidades faltan para alcanzar el mínimo exigido, y compara la suma de estos faltantes contra la cantidad de posiciones aún disponibles en la contraseña; si dicha suma supera las posiciones restantes, la rama actual se descarta de inmediato, sin generar ninguno de sus descendientes. La construcción de la contraseña se realiza de manera incremental, carácter por carácter, actualizando en cada paso los contadores de composición y el último carácter agregado, con el fin de aplicar también la restricción de no repetir caracteres consecutivamente.

Se implementaron además dos modalidades de salida para el algoritmo con poda: una que genera y almacena explícitamente todas las contraseñas válidas, y otra que únicamente cuenta el número de soluciones sin almacenarlas, pensada para instancias donde el número de contraseñas válidas resulta demasiado grande para mantenerse en memoria. Con fines de comparación, tal como lo exige el enunciado, se implementó también una versión sin poda, independiente de la versión con poda, que recorre el árbol completo de posibilidades hasta llegar a cadenas de longitud n y solo entonces verifica si cumplen la política, permitiendo así contrastar el número de nodos explorados frente a la versión con poda.

6. Pseudocódigos

```

/*
PSEUDOCÓDIGO DE LA FUNCIÓN DE GENERACIÓN Y VERIFICACION DE CONTRASEÑAS FB

alfabeto[a-z]

generarContraseña(contraseña, tamañoContraseña, alfabeto, hashObjetivo):

si funcionHash(contraseña) == hashObjetivo:
    imprimir "La contraseña es: " + contraseña
    retornar

Si tamañoContraseña == longitud(contraseña):
    retornar

para cada caracter en alfabeto:
    // llamada recursiva para generar todas las combinaciones posibles de contraseñas
    generarContraseña(contraseña+caracter, tamañoContraseña, alfabeto, hashObjetivo)

*/

```

```

/*
PSEUDOCODIGO GENERAL DEL MODULO BT (BACKTRACKING)

resolverBacktracking(prefijo, contadores, politica, alfabeto):
    si tamaño(prefijo) == politica.n:
        si contadores cumplen los minimos de la politica:
            guardar prefijo como solucion
            retornar

    si NO esFactible(prefijo, contadores, politica):
        retornar // PODA: se abandona la rama sin explorar descendientes

    para cada caracter en alfabeto:
        si caracter == ultimoCaracter y politica prohíbe repetidos:
            continuar
        actualizar contadores segun el tipo de caracter
        resolverBacktracking(prefijo + caracter, contadores, politica, alfabeto)

*/

```

7. Implementación

Para la solución del módulo de **Fuerza Bruta**, se hizo uso de la modularidad de código a través de funciones orquestadas desde el `main()`. En esta, se creó un menú para incentivar la optimización de recursos computacionales que no sean necesarios (se evita correr pruebas en caso de que no se quieran hacer). Adicionalmente, para la transformación y comprobación de los hashes de las contraseñas se importó una biblioteca de terceros llamada “`picosha.h`” que implementa la función del `hash256`, evitando así implementarla desde cero. Finalmente, para darle personalización y validación procedimental al código, se creó una semilla (la suma del valor `ascii` de los 3 apellidos en minúsculas) con la que, con una fórmula brindada por el docente, se crearon instancias de `testing`.

Para la solución del módulo de **Backtracking**, el código se organizó en archivos independientes según su responsabilidad, con el fin de facilitar tanto el mantenimiento como la escritura de pruebas automatizadas por separado. El archivo `bt_policy.hpp` concentra el cálculo de la semilla del equipo y la derivación de los parámetros de la política de seguridad a partir de dicha semilla, además de una función de utilidad para clasificar caracteres según su tipo. El archivo `bt_backtrack.hpp` contiene exclusivamente la función de factibilidad y el algoritmo de `backtracking` con poda, en sus dos modalidades de generación y conteo de soluciones. De manera independiente, el archivo `bt_sinpoda.hpp` implementa la versión sin poda utilizada como punto de comparación, sin depender del archivo anterior, de modo que ambas versiones puedan evaluarse de forma aislada.

Finalmente, `bt_main.cpp` orquesta ambos algoritmos desde un menú interactivo, similar en su estructura al del módulo de Fuerza Bruta: una primera opción permite validar la implementación contra la instancia de referencia común del curso, y una segunda opción ejecuta las cinco variantes de dificultad solicitadas, comparando en cada una la versión con poda contra la versión sin poda, verificando

que el número de soluciones coincida entre ambas, y registrando los resultados en un archivo CSV. Dado que durante la experimentación se identificaron instancias cuyo espacio de búsqueda resulta intratable incluso en modo conteo, se incorporó un límite opcional y configurable de nodos visitados, que el usuario puede activar o desactivar antes de ejecutar las pruebas, permitiendo así obtener resultados parciales claramente documentados en aquellos casos donde completar la búsqueda no resulta computacionalmente viable.

9. Análisis de complejidad

En el módulo de **Fuerza Bruta**, para una longitud fija n y un alfabeto de tamaño $|\Sigma|$, existen $|\Sigma|^n$ candidatos posibles. Como el algoritmo debe generar y verificar cada candidato hasta encontrar el objetivo o agotar el espacio, su complejidad temporal en el peor caso es $O(|\Sigma|^n)$, considerando el cálculo del hash como una operación constante respecto de n para las longitudes acotadas utilizadas en el experimento.

En cuanto al espacio, la implementación de Fuerza Bruta no necesita almacenar simultáneamente todos los candidatos. Es suficiente mantener el candidato que se está evaluando, junto con las estructuras auxiliares necesarias para calcular su hash y controlar la enumeración. Por tanto, para una longitud máxima n , el espacio utilizado es $O(n)$.

Para **Backtracking**, el peor caso mantiene igualmente un crecimiento exponencial. Si las restricciones no permiten realizar poda efectiva, el algoritmo puede recorrer prácticamente todo el árbol de búsqueda, cuyo número de nodos hasta profundidad n está dado por $\sum_{k=0}^n |\Sigma|^k$. En consecuencia, para el alfabeto de 69 símbolos, la complejidad temporal del peor caso es $O(69^n)$.

El mejor caso de Backtracking ocurre cuando las restricciones permiten detectar la inviabilidad de una rama muy temprano, reduciendo considerablemente el número de estados visitados. El peor caso ocurre cuando las restricciones son poco restrictivas y prácticamente todas las ramas continúan siendo factibles. El comportamiento promedio depende de la política específica y de cómo sus restricciones interactúan con el orden de exploración. Especialmente, la profundidad máxima del árbol es n , por lo que la pila de recursión y el estado parcial requieren $O(n)$ espacio, sin considerar el almacenamiento de las soluciones si estas deben conservarse explícitamente.

10. Casos de prueba

Para validar el funcionamiento del algoritmo de Backtracking se hicieron casos de prueba orientados a comprobar tanto la correcta generación de contraseñas como el cumplimiento de las restricciones definidas para la política de seguridad. La política utilizada establece una longitud de 8 caracteres, con un mínimo de 3 letras minúsculas, 2 letras mayúsculas, 2 dígitos y 1 símbolo, además de impedir caracteres idénticos consecutivos.

Los casos de prueba se enfocaron principalmente en los siguientes aspectos:

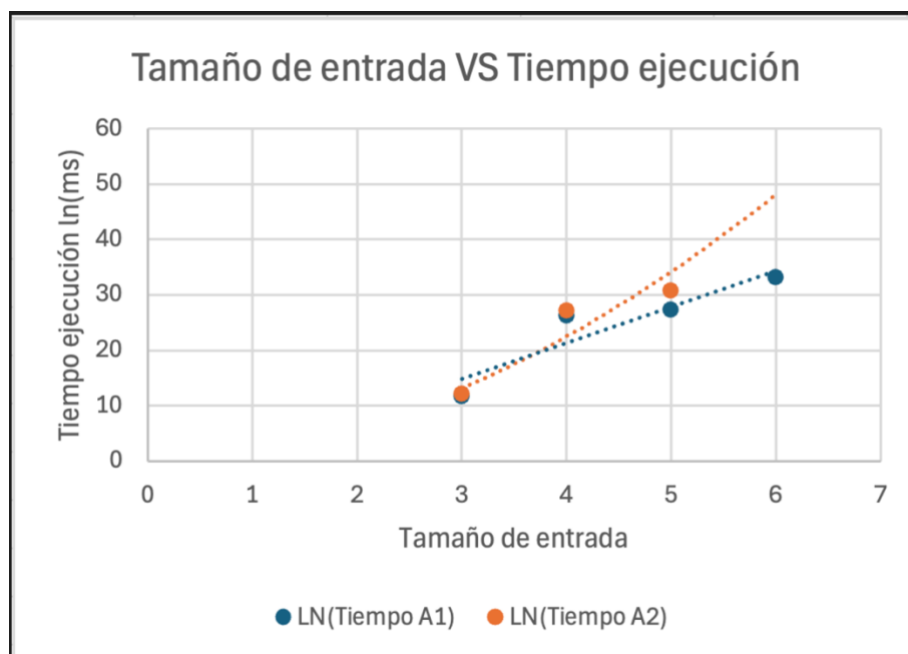
- Validación de restricciones: comprobar que las contraseñas generadas cumplen con las cantidades mínimas requeridas para cada tipo de carácter y con la restricción de no repetir caracteres consecutivamente.
- Generación de soluciones: verificar que el algoritmo sea capaz de generar correctamente las contraseñas que cumplen la política establecida.
- Conteo de soluciones: comprobar que la modalidad de conteo produzca el mismo número de soluciones que la modalidad de generación.
- Comparación con y sin poda: ejecutar ambas versiones del algoritmo sobre las mismas instancias y verificar que obtengan el mismo número de soluciones, permitiendo comparar posteriormente la cantidad de nodos explorados y el tiempo de ejecución.
- Casos de gran tamaño: utilizar un límite configurable de nodos visitados para aquellas instancias cuyo espacio de búsqueda resulta demasiado grande para completar la exploración en un tiempo razonable. Esto permite registrar resultados parciales sin comprometer la ejecución del programa.

Estos casos permiten verificar tanto la correctitud funcional del algoritmo como el beneficio de la estrategia de poda. En particular, se espera que ambas versiones encuentren las mismas soluciones, pero que la versión con poda explore una cantidad menor de estados y, por tanto, presente un menor costo computacional cuando las restricciones permiten descartar ramas de manera temprana.

11. Experimentación

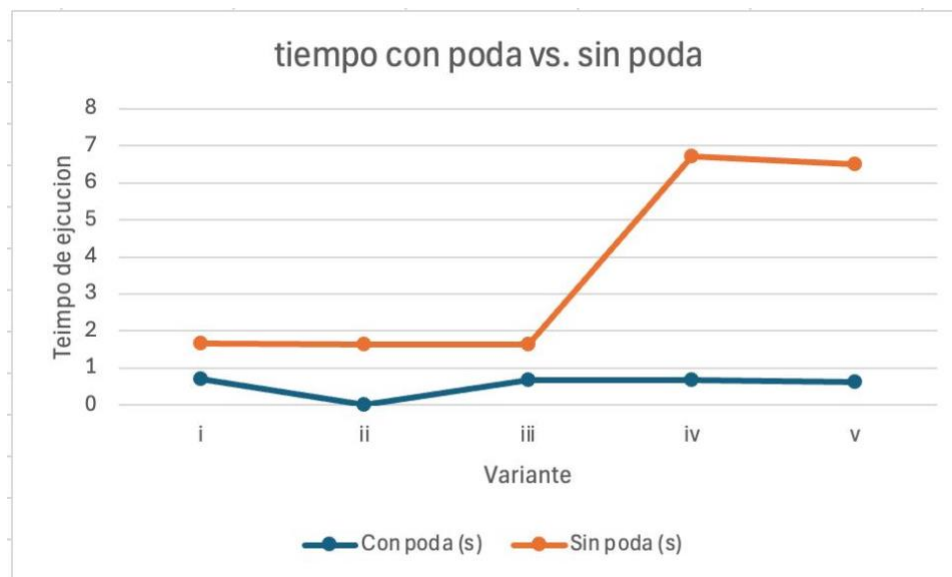
Para Fuerza Bruta:

Alfabeto	Longitud	Tiempo de ejecución (μ s)	candidato evaluado	resultado
abcdefghijklmnopqrstuvwxyz	3	113123	mmm	Encontrado
abcdefghijklmnopqrstuvwxyz	4	2.57022e+06	mmm	Encontrado
abcdefghijklmnopqrstuvwxyz	5	6.8379e+07	mmmm	Encontrado
abcdefghijklmnopqrstuvwxyz	6	2.45415e+09	mmmmm	Encontrado
abcdefghijklmnopqrstuvwxyz123456789	3	173843	mmm	Encontrado
abcdefghijklmnopqrstuvwxyz123456789	4	6.11412e+06	mmmm	Encontrado
abcdefghijklmnopqrstuvwxyz123456789	5	2.20411e+08	mmmmm	Encontrado



Backtracking tiempo mostrado en segundos con un limite de 20.000.000 de nodos

Column1	Tiempo con poda	Tiempo sin poda
Politica completa n=8	0.713586s	1.65853s
Politica completa n=6	0.0000006s	1.6286s
Politica completa n=10	0.681604s	1.62797s
Política relajada minLower=1	0.664684s	6.71278s
política sin ninguna restricción de composición	0.633304s	6.5049s



12. Resultados

Para **Fuerza Bruta**, en las pruebas realizadas se observó un incremento considerable en el tiempo de ejecución a medida que aumentó la longitud de la contraseña. Para el alfabeto A1, el tiempo pasó de **113.123 ms** con longitud 3 a aproximadamente **40.9 minutos** con longitud 6. En el caso de A2, aumentó de **173,843 ms** para longitud 3 a aproximadamente **220,4 segundos** para longitud 5. En todas las instancias evaluadas, la contraseña objetivo fue encontrada correctamente, confirmando el funcionamiento del algoritmo. Estos resultados evidencian que pequeños incrementos en la longitud generan aumentos muy grandes en el costo de la búsqueda, especialmente para el alfabeto de mayor tamaño.

Para **Backtraking**, se pudo evidenciar que, al limitar la búsqueda de los árboles a 20 millones de nodos, hay un tiempo de respuesta considerablemente distinto en cuanto a segundos, ya que por ejemplo en la búsqueda sin restricciones de composición, es mucho más tiempo sin poda que con poda, teniendo con poda, una respuesta de 0.7 segundos, mientras que sin poda, tenemos una respuesta de 6.5 segundos. Esto es evidencia de que con números mucho mas grandes, el algoritmo con poda siempre va a ser mucho mas eficiente, siempre y cuando las restricciones estén bien definidas.

13. Análisis de resultados

Los resultados obtenidos son consistentes con la complejidad teórica $O(\Sigma^n)$ de fuerza bruta. El crecimiento observado en la gráfica de **ln(tiempo)**

frente a la longitud presenta una tendencia aproximadamente lineal, lo que respalda el comportamiento exponencial del tiempo de ejecución respecto a n . Además, A2 presenta tiempos superiores a A1 para longitudes comparables, debido a que su alfabeto contiene **35 símbolos frente a los 26 de A1**, aumentando la cantidad de candidatos que deben ser explorados. El crecimiento se vuelve especialmente evidente en las longitudes mayores, donde el costo de evaluar exhaustivamente el espacio comienza a ser poco manejable, confirmando experimentalmente el límite práctico de este enfoque.

14. Comparación algorítmica (fuerza bruta vs. diccionario; con poda vs. sin poda)

La fuerza bruta explora sistemáticamente todas las combinaciones posibles de un alfabeto, garantizando encontrar la contraseña dentro del espacio de búsqueda, pero con un alto costo computacional. En cambio, la “búsqueda por diccionario” prueba únicamente contraseñas comunes, reduciendo considerablemente el número de intentos y siendo más eficiente cuando la contraseña es predecible. Este enfoque puede verse como una estrategia “similar a la codiciosa”, al priorizar candidatos que tienen mayor probabilidad de ser la solución, aunque no constituye formalmente un algoritmo Greedy.

El **Backtracking** explora las posibles combinaciones mediante la construcción progresiva de soluciones, descartando aquellas ramas que no pueden cumplir las restricciones, lo que reduce considerablemente el espacio de búsqueda y el costo computacional. En cambio, el **Backtracking sin poda** recorre todas las posibilidades y verifica las restricciones únicamente al final, siendo menos eficiente. Por esto, la poda permite que Backtracking sea más eficiente cuando las restricciones están bien definidas, aunque en el peor caso ambos pueden presentar un comportamiento exponencial.

15. Conclusiones.

La implementación de Fuerza Bruta y Backtracking permitió comprobar cómo la estrategia de búsqueda afecta el rendimiento de un algoritmo. En Fuerza Bruta se evidenció un crecimiento exponencial del tiempo de ejecución a medida que aumentaba la longitud de las contraseñas, confirmando su complejidad $\mathcal{O}(|\Sigma|^n)$. Por otro lado, Backtracking demostró ser más eficiente al usar poda para descartar ramas que no pueden generar soluciones válidas. En las pruebas

realizadas, la versión con poda obtuvo tiempos mucho menores que la versión sin poda, demostrando el beneficio de aprovechar las restricciones del problema. En conclusión, aunque ambos métodos pueden presentar un comportamiento exponencial en el peor caso, el uso de poda en Backtracking permite reducir significativamente el espacio explorado y mejorar el rendimiento. La práctica permitió relacionar la complejidad teórica con resultados obtenidos en una implementación real.

17. Uso de herramientas de IA

Para Fuerza Bruta:

The screenshot shows the Claude AI web interface. On the left is a sidebar with navigation options: New, Projects, Artifacts, Code (with an Upgrade button), and Customize. Below these are 'Chats and tasks' with a list of recent chats, including 'Módulo de fuerza bruta: análisis y documentación'. The main chat area has a title 'Módulo de fuerza bruta: análisis y documentación' and a 'Free plan Upgrade' button. A PDF document is attached to the chat. The chat history shows a greeting from Claude, followed by a user prompt asking Claude to act as a security analyst and summarize a PDF. Claude responds with a detailed summary and a list of tasks for the 'Brute Force Module'.

Hola, actúa como un súper analista y resumidor de pdf's. Estoy encargado de hacer el modulo de Fuerza Bruta, puedes resumirme toda la información importante para saber bien qué es lo que debo desarrollar y que documentación debo hacer por favor.

¡Claro! Aquí tienes un resumen completo y organizado de todo lo que necesitas saber y desarrollar para el **Módulo FB (Fuerza Bruta)**.

1. Tu rol y la pregunta central

Actúas como un **atacante/analista de seguridad** que no conoce una contraseña y debe medir experimentalmente cuánto cuesta encontrarla probando **todas** las combinaciones posibles (sin usar ninguna propiedad estructural del problema para descartar candidatos).

2. Qué debes construir (12 actividades mínimas)

- Modelamiento:** definir el espacio de búsqueda — alfabeto(s), rango de longitudes, función de verificación (comparar hashes).
- Entradas/salidas:** entrada = hash objetivo SHA-256, alfabeto, longitud máxima; salida = contraseña encontrada o "no encontrado dentro del espacio".
- Restricciones:** qué alfabetos/longitudes se tratan y por qué son tratables computacionalmente.

Write a message...

Claude is AI and can make mistakes. Please double-check responses. Sonnet 5 Medium

Claude

New

Projects

Artifacts

Code Upgrade

Customize

Chats and tasks

Módulo de fuerza bruta: análisis y documentación

Optimización de cartera de préstamos

Análisis comparativo de algoritmos

Resumen de libro sobre coreanos Z

Búsqueda flexible de nombres con

Layered client-server project struc

Bounded set problems explained

Differential notation: dx versus ds

PDF translation from Spanish to En

Nand2Tetris space invaders game o

Jack compiler syntax errors in Fle

Design

salmon · Free

Módulo de fuerza bruta: análisis y documentación

Free plan Upgrade

Share

¿Qué estructura de carpetas/README se generaría para un proyecto de fuerza bruta que genere contraseñas y verifique si coinciden con la estructura de carpetas/README del repositorio?

En este momento estoy un poco estancado para comenzar porque aún no entiendo muy bien el modus operandi en relación con lo de los hashes. Ya tengo el algoritmo que va a seguir la función por fuerza bruta pero no comprendo muy bien cómo funciona la interacción, según LO ENTIENDO AHORA ES: 1. creo una posible contraseña, 2. Saco su hash con la función de hashing definida por las semillas y una librería ya posiblemente establecida, 3. Verificar que coincide. Pero entonces no comprendo muy bien porque se recibe el hash de la contraseña que quiero como entrada. Me puedes explicar en general cómo funcionaría el uso del hash...

Thought for 10s

¡Buena pregunta! Es normal confundirse aquí porque hay **dos roles distintos** mezclados en el enunciado, y es clave separarlos mentalmente.

Primero, aclaremos un malentendido importante

Dijiste "el hash de la contraseña que saco con la función de hashing definida por las semillas". Ahí hay una confusión: **la semilla NO define ninguna función de hashing**. La semilla (Sección 9.1) solo sirve para **generar las contraseñas de prueba** (los "candidatos objetivo" que tu equipo va a intentar romper). El algoritmo de hashing (SHA-256) es **siempre el mismo, fijo y estándar** — no cambia según la semilla ni según nada. Viene de la librería (`pcsha2.h` o similar) y su comportamiento no lo diseñas tú.

Los dos roles que coexisten en esta práctica

Rol 1: "Creador de instancias" (tú generando los casos de prueba)

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses. Sonnet 5 Medium

Para Backtracking:

Claude

Nuevo

Proyectos

Artefactos

Código Actualizar

Personalizar

Chats y tareas

Error de compilación en generación de semilla

Explicación algoritmo

Preparar exposición c

Guion de reducción d

Family tree genealog

Turing Machine Table

Double Integral step

Conversión de Grami

Context-Free Gramm

MySQL Reverse Engi

Coq Code README C

Implementing Coq Fu

Using JSCoq, the we

Implementing Statist

Diseño

Jc ·

Error de compilación en generación de semilla

Plan gratuito Actualizar

Compartir

Buenas, ando realizando un algoritmo el cual se encarga de la generación de contraseñas válidas bajo restricciones mediante Backtracking, ando haciendo el apartado que se encarga de calcular la semilla de antes de correr cualquier experimento.

El problema que tengo es que durante la compilación obtuve un error que me impidió la generación de la semilla, siendo el error el siguiente: `initial value of reference to non-const must be an lvalue`

Quiero que me expliques porque ocurre ese tipo de errores, no me des la solución, simplemente explícame la lógica detrás del error.

¡Buenas! Con gusto te explico la lógica detrás de este error.

El concepto central: lvalue vs rvalue

En C++, toda expresión tiene una "categoría de valor". Las dos que te interesan aquí son:

- lvalue** (locator value): es algo que tiene una ubicación de memoria identificable y persistente. Básicamente, algo a lo que le puedes "poner nombre" y tomar su dirección. Ejemplos: una variable declarada (`int x;`), un elemento de un arreglo (`arr[3]`), lo que retorna una función que devuelve una referencia.
- rvalue**: es un valor temporal, que no tiene una dirección de memoria estable ni un nombre propio. Existe solo momentáneamente para ser usado en una expresión. Ejemplos: el resultado de `x + y`, un literal como `5`, el retorno de una función que devuelve por valor (no por referencia).

Por qué las referencias no-const exigen un lvalue

Una referencia (`int&`, `std::string&`, etc.) es, conceptualmente, un alias de una variable existente. Cuando declaras:

```
cpp
int& ref = algo;
```

+ Escribe un mensaje...

Claude es una IA y puede cometer errores. Comprueba las respuestas. Sonnet 5 Medio

18. Contribución individual de los integrantes

Simón Mazo Gómez: Diseño e implementación del algoritmo de fuerza bruta.

Johan Alejandro Peña Rivera: Diseño e implementación del algoritmo de Backtracking

Mateo Gomez Giraldo: Casos de prueba y verificación experimental.